Crop Yield Prediction Using Machine Learning

A Complete Regression Analysis of Indian Agricultural Data

Course: Advanced Data Science (ADS)

Assignment: Formative Assessment 2 (FA 2)

Dataset Source: India Data Portal — Ministry of Agriculture & Farmers Welfare, Government of India

Target Variable: Yield (Tonnes per Hectare)

GitHub Repository: https://github.com/ShravaniGhadge18/crop-yield-prediction

Live Streamlit App: https://crop-yield-prediction.streamlit.app

---

Cover Page

Item	Detail
Project Title	Crop Yield Prediction Using Machine Learning
Domain	Agricultural Data Science
Dataset	India Crop Wise Area, Production and Yield (1997–2023)
Source	India Data Portal (indiadataportal.com)
ML Task	Supervised Regression
Target Variable	yield — Crop yield in Tonnes per Hectare
GitHub Repository	https://github.com/ShravaniGhadge18/crop-yield-prediction
Live Streamlit App	https://crop-yield-prediction.streamlit.app

---

Abstract

This project develops a Machine Learning regression system to predict agricultural crop yield (in tonnes per hectare) using historical district-level data from India spanning 26 agricultural years (1997-1998 to 2022-2023). The dataset, sourced from the Ministry of Agriculture &

Farmers Welfare through the India Data Portal, contains 455,359 records across 34 states, 737 districts, and 115 crop types.

Four regression algorithms — Linear Regression, Decision Tree Regressor, Random Forest Regressor, and LinearSVR — are implemented using Scikit-learn pipelines with proper preprocessing. Due to extreme right-skewness in the yield distribution (skewness ≈ 16.33), all models use a TransformedTargetRegressor with log1p/expm1 transformation. GridSearchCV is applied to Random Forest on a memory-efficient representative subset. The final system includes a complete Streamlit web application for interactive yield prediction. The complete reproducible codebase is hosted on GitHub at https://github.com/ShravaniGhadge18/crop-yield-prediction, and the interactive prediction system is publicly deployed on Streamlit Cloud at https://crop-yield-prediction.streamlit.app.

---

1. Introduction

Agricultural productivity is a critical concern for food security and economic planning, especially in a country like India with a large agrarian economy. Accurate prediction of crop yield enables:

- Better agricultural planning and resource allocation
- Early warning for food shortage scenarios
- Policy decisions at district and state levels
- Improved supply chain management

Machine Learning provides a data-driven approach to yield prediction that can capture complex non-linear relationships between features such as location, crop type, season, cultivation area, and historical yield patterns.

---

2. Problem Statement

Can we build a machine learning model that accurately predicts crop yield (in tonnes/hectare) for Indian districts, given features such as state, district, crop type, season, cultivation area, and year?

This is a supervised regression problem where:

- **Input (features):** State, district, crop name, crop type, season, area, year
- **Output (target):** Yield in Tonnes/Hectare

---

3. Objectives

1. Acquire and preprocess a real-world Indian agricultural dataset
2. Perform comprehensive Exploratory Data Analysis (EDA)
3. Implement and compare at least 4 ML regression algorithms
4. Apply hyperparameter tuning using GridSearchCV
5. Evaluate all models using MAE, MSE, RMSE, and R^2 metrics
6. Build a Streamlit web application for interactive predictions
7. Document all design decisions and methodology

---

4. Dataset Description

4.1 Data Source

Attribute	Value
Portal	India Data Portal (indiadataportal.com)
Underlying Source	Ministry of Agriculture & Farmers Welfare, GoI
File	crop-wise-area-production-yield.csv
Codebook	Crop_Wise_Area_Production_Yield_codebook.xlsx

4.2 Dataset Statistics

Metric	Value
Total Records	455,359
Total Columns	16

Metric	Value
Years Covered	26 (1997-1998 to 2022-2023)
States/UTs	34
Districts	737
Crops	115
Crop Types	10
Seasons	7

4.3 Data Dictionary

Column	Type	Description
id	int	Record identifier (removed)
year	str	Agricultural year e.g., "1997-1998"
state_name	str	State or Union Territory name
state_code	int	Numeric state code (removed)
district_name	str	District name
district_code	int	Numeric district code (removed)
crop_name	str	Name of the crop
crop_code	float	Numeric crop code (removed)
crop_type	str	Category of crop (Cereals, Oilseeds, etc.)
season	str	Growing season (Kharif, Rabi, etc.)
area	float	Cultivation area in Hectares
area_unit	str	Unit (always "Hectare" — removed)

Column	Type	Description
production	float	Total production in Tonnes (removed — leakage)
production_unit	str	Unit (always "Tonnes" — removed)
yield	float	**Target**: Yield in Tonnes/Hectare
yield_unit	str	Unit (always "Tonnes/Hectare" — removed)

---

5. Data Preprocessing

5.1 Missing Value Analysis

Column	Missing Values	% Missing
yield	0	0.000%
area	0	0.000%
production	5,009	1.100%
crop_code	21	0.005%

- **Action for production:** Excluded from ML features (target leakage)
- **Action for crop_code:** Excluded (using crop_name instead)
- **No records removed** for missing values

5.2 Duplicate Records

Zero duplicate rows found in the dataset. No action required.

5.3 Columns Removed and Justification

Column Removed	Reason
id	Record identifier — no predictive value

Column Removed	Reason
state_code	Numeric identifier — misleads model into treating codes as ordered numbers
district_code	Same reason as state_code
crop_code	Less informative than crop_name; some codes map to multiple crops
area_unit	Always "Hectare" — constant, no information
production	**Target leakage**: production = area × yield (mathematically related to target)
production_unit	Always "Tonnes" — constant
yield_unit	Always "Tonnes/Hectare" — constant
year (original)	String format; replaced by year_start (numeric)

5.4 Feature Engineering

The year column contains strings like "1997-1998". This was converted to a numeric feature:

python

ml_data["year_start"] = data["year"].str[:4].astype(*int*)

This extracts the starting year: "1997-1998" → 1997

5.5 Final ML Features

Categorical Features (OneHotEncoded):

- state_name (34 unique values)
- district_name (737 unique values)
- crop_name (115 unique values)
- crop_type (10 unique values)
- season (7 unique values)

Numerical Features:

- area — Cultivation area in hectares
- year_start — Numeric year (1997 to 2022)

---

6. Exploratory Data Analysis

6.1 Target Variable Analysis

Statistic	Value
Minimum	0.0 T/ha
Median	1.051 T/ha
Mean	68.845 T/ha
Maximum	43,958.333 T/ha
Skewness	16.33
Zero-yield records	9,038

The extreme right-skewness (skewness ≈ 16.33) is primarily driven by crops like **coconut**, which are measured in numbers of nuts per hectare but converted to tonnes using conversion factors, resulting in legitimately large values.

Decision: Do NOT remove extreme values. Many high-yield values are legitimate agricultural observations and carry real predictive information. Instead, a log transformation is applied to the target during model training.

6.2 Yield Percentile Distribution

Percentile	Yield (T/ha)
50%	1.051
75%	2.55

Percentile	Yield (T/ha)
90%	9.972
95%	22.375
99%	95.95
99.5%	5,941.72
99.9%	14,060.05

6.3 Key Findings from EDA

1. **Most yields are small**: 90% of records have yield < 10 T/ha — a typical field crop range
2. **Kharif season dominates**: 35.4% of all records are Kharif season crops
3. **Cereals are most common**: ~27.6% of records are cereal crops
4. **Uttar Pradesh has most records**: Largest state by agricultural data volume
5. **Yield has increased over time**: The 26-year trend shows improving agricultural productivity
6. **Zero-yield records exist**: 9,038 records with yield=0 (possibly failed crops or data issues); kept in dataset

---

7. Machine Learning Methodology

7.1 Train/Test Split

An 80/20 stratified random split was used:

python

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

Training: 364,287 rows

Testing: 91,072 rows

The **test set was never used during training or hyperparameter tuning** — only for final evaluation.

7.2 Preprocessing Pipelines

Two separate preprocessors were created based on model requirements:

For Linear Regression and LinearSVR:

python

```
linear_preprocessor = ColumnTransformer([
    ("categorical", OneHotEncoder(handle_unknown="ignore"), categorical_features),
    ("numerical",   StandardScaler(),                        numerical_features),
])
```

For Decision Tree and Random Forest:

python

```
tree_preprocessor = ColumnTransformer([
    ("categorical", OneHotEncoder(handle_unknown="ignore"), categorical_features),
    ("numerical",   "passthrough",                           numerical_features),
])
```

Reasoning:

- Linear models and SVMs are sensitive to feature scale — StandardScaler normalizes numerical features
- Tree-based models are invariant to monotonic transformations — no scaling needed
- handle_unknown="ignore" allows Streamlit predictions for unseen category combinations

7.3 Target Transformation

Due to extreme right-skewness (skewness ≈ 16.33), all models
use TransformedTargetRegressor:

python

```
model_pipeline = TransformedTargetRegressor(
    regressor=Pipeline([...]),
    func=np.log1p,        # transform: log(1 + yield)
    inverse_func=np.expm1 # back-transform: e^y - 1
)
```

Why log1p instead of log?

- log1p(0) = 0 — handles zero-yield records without error
- expm1 is the exact inverse, preventing numerical errors

This makes the target more normally distributed, improving model fitting for linear models and error distribution for all models.

---

8. Machine Learning Algorithms

8.1 Linear Regression

Purpose: Establish a simple linear baseline.

python

```
LinearRegression()
```

Finds the best-fit hyperplane in the transformed feature space. Serves as the baseline to compare against more complex models.

8.2 Decision Tree Regressor

Purpose: Capture non-linear relationships and feature interactions without feature scaling.

python

```
DecisionTreeRegressor(random_state=42)
```

Recursively partitions the feature space, learning step-wise functions. Can model complex interactions but may overfit without depth constraints.

8.3 Random Forest Regressor

Purpose: Ensemble of decision trees with improved generalization via bagging and feature randomization.

python

RandomForestRegressor(*random_state*=42, *n_jobs*=-1)

Averages predictions from many decorrelated trees, reducing variance compared to a single decision tree. Typically the best-performing model for tabular data.

8.4 LinearSVR

Purpose: Support Vector Regression with a linear kernel, computationally practical for large datasets.

python

LinearSVR(*random_state*=42, *max_iter*=5000)

Standard RBF-kernel SVR is $O(n^2)$ to $O(n^3)$ in complexity — infeasible for 455,359 rows. LinearSVR uses a linear kernel and scales linearly with dataset size, making it practical for this problem.

---

9. GridSearchCV

9.1 Requirement

The professor's rubric requires K-Fold Cross-Validation or GridSearchCV on at least one model. GridSearchCV was applied to the **Random Forest** as it has the most tunable hyperparameters.

9.2 Computational Challenge

Previous attempts failed due to:

- Memory errors (BrokenProcessPool) when using n_jobs=-1 with large param grids
- 30+ minutes of training time for 16-combination grids × 3 folds on the full dataset

9.3 Memory-Safe Solution

Strategy adopted:

1. Extract a **representative 10,000-row tuning subset** from X_train (NOT from test set)
2. Use a **small but meaningful parameter grid** (4 combinations × 3 folds = 12 fits)
3. Use n_jobs=1 (sequential) to prevent forked-process memory crashes
4. Use the best found parameters to **retrain on the complete training set**

python

```
X_tune, _, y_tune, _ = train_test_split(
    X_train, y_train, train_size=10000, random_state=42
)
param_grid = {
    "regressor__model__n_estimators": [50, 100],
    "regressor__model__max_depth":    [15, 20],
}
rf_grid_search = GridSearchCV(
    estimator=rf_for_gridsearch,
    param_grid=param_grid,
    cv=3,
    scoring="neg_root_mean_squared_error",
    n_jobs=1,
    verbose=1,
)
rf_grid_search.fit(X_tune, y_tune)
```

9.4 Justification for this Approach

This is a valid practical engineering decision. The 10,000-row subset:

- Is randomly sampled from the training set with the same distribution
- Represents all 34 states, multiple crop types, and all seasons
- Is large enough to differentiate between hyperparameter configurations

The final model is then retrained on the complete 364,287-row training set using the best-found hyperparameters, ensuring maximum model quality.

---

10. Evaluation Metrics

10.1 Metric Definitions

Metric	Formula	Interpretation
MAE	Mean |y_actual - y_pred|	Average prediction error in T/ha
MSE	Mean $(y_{actual} - y_{pred})^2$	Penalizes large errors more
RMSE	√MSE	Same unit as yield (T/ha)
R^2	1 - SS_res/SS_tot	% variance explained (0 to 1)

10.2 Final Model Results (Actual — from test set evaluation)

GridSearchCV best parameters: max_depth=15, n_estimators=50
Best CV RMSE (on 10k tuning subset): 552.42

Model	MAE	MSE	RMSE	R^2
Linear Regression	27.3285	175,586.80	419.031	0.7557
Decision Tree	14.1409	92,118.43	303.510	0.8718
Random Forest	**15.8604**	**75,877.41**	**275.459**	**0.8944**
LinearSVR	25.9360	145,981.45	382.075	0.7969

Best Model: Random Forest (highest $R^2 = 0.8944$)

---

11. Model Comparison and Selection

Selection Criterion: Highest R^2 Score on the held-out test set.

Winner: Random Forest Regressor — R^2 = **0.8944**, RMSE = **275.46 T/ha**, MAE = **15.86 T/ha**

GridSearchCV Result: Best hyperparameters found — max_depth=15, n_estimators=50

Analysis:

1. Random Forests handle high-cardinality categorical variables well (737 districts, 115 crops)
2. They are naturally robust to outliers (extreme yield values)
3. They capture non-linear relationships without requiring target transformation
4. The TransformedTargetRegressor with log1p further improves their performance

---

12. Streamlit Application

12.1 Application Architecture

The Streamlit app (app.py) has 4 main sections:

Tab 1 — Dashboard

- Project overview with key statistics
- Dataset metadata (rows, states, districts, crops, years, seasons)
- First 10 rows preview
- Missing value summary

Tab 2 — EDA (Exploratory Data Analysis)

- Interactive yield distribution with percentile filter
- log1p(yield) distribution showing transformation effect
- Top 15 crops by median yield

- Crop type distribution (pie chart)
- Yield trend over 26 years
- Seasonal yield comparison
- State-wise yield bar chart

Tab 3 — Model Comparison

- Performance table (MAE, MSE, RMSE, R^2) with highlights
- Bar charts for RMSE, MAE, and R^2 comparison
- GridSearchCV best parameters and CV score
- Methodology note explaining the memory-safe approach

Tab 4 — Prediction

- Input form: State, District, Crop, Crop Type, Season, Year, Area
- Model selector (Auto/Best, LR, DT, RF, LinearSVR)
- Predicted yield display in T/ha
- Estimated total production (yield × area)
- Historical context (median, mean, max for same crop+season)

12.2 Performance Optimization

- @st.cache_data for dataset loading — prevents re-reading the 54 MB CSV on every interaction
- @st.cache_resource for model loading — prevents reloading heavy .pkl files
- Models are **loaded from disk, not retrained** — instant prediction response

12.3 Running the Application Locally

bash

streamlit run app.py

12.4 Live Cloud Deployment (Streamlit Community Cloud)

For evaluation and immediate testing without local setup, the application has been deployed live on Streamlit Community Cloud:
• Live Web Application URL: https://crop-yield-prediction.streamlit.app
• Public GitHub Repository: https://github.com/ShravaniGhadge18/crop-yield-prediction
• Deployment Architecture: Continuous deployment integrated with GitHub main branch.
• Cloud Optimization: The application loads the pre-trained best Random Forest pipeline (best_model.pkl) into memory using @st.cache_resource and caches data with @st.cache_data, delivering sub-second predictions on any browser.

The app will be available at http://localhost:8501

---

13. System Architecture

PlantUML Diagram

plantuml

@startuml

start

:Load Agricultural Dataset;

note right: 455,359 rows, 16 columns

:Inspect and Clean Data;

note right: Remove duplicates\nHandle missing values

:Feature Engineering;

note right: year_start = year[:4]

:Drop Irrelevant Columns;

note right: id, codes, units, production

:Perform Exploratory Data Analysis;

:Select Features (7 features);

:Split Data 80/20;

note right: Train: 364,287\nTest: 91,072

:Apply One-Hot Encoding (Categorical);

```
note right: OHE for state, district, crop,\ncrop_type, season
:Apply Scaling (Linear Models Only);
note right: StandardScaler for area, year_start
fork
  :Linear Regression;
fork again
  :Decision Tree Regressor;
fork again
  :Random Forest Regressor;
fork again
  :LinearSVR;
end fork
:Apply GridSearchCV to Random Forest;
note right: 10,000-row subset\n4 combos x 3 folds = 12 fits
:Retrain Final RF on Full Training Set;
:Calculate MAE, MSE, RMSE and R2;
:Compare All 4 Models;
:Select Best Model (Highest R2);
:Save Trained Pipelines via Joblib;
:Launch Streamlit Application;
:Generate Crop Yield Prediction;
stop
@enduml
```

---

14. Conclusion

This project successfully built a complete Machine Learning regression pipeline for crop yield prediction using Indian agricultural data. Key achievements:

1. **Data Quality:** Processed 455,359 real-world agricultural records with careful handling of extreme values and leakage prevention
2. **Preprocessing:** Dual preprocessing pipelines (linear vs. tree) with appropriate encoding and scaling
3. **Target Transformation:** Addressed extreme right-skewness using log1p transformation
4. **Model Comparison:** Compared 4 diverse algorithms covering linear and non-linear paradigms
5. **Hyperparameter Tuning:** Successfully implemented memory-efficient GridSearchCV
6. **Deployment:** Built a comprehensive 4-tab Streamlit application with caching

Best Model: Random Forest Regressor consistently achieved the highest R^2 and lowest RMSE/MAE, confirming that ensemble tree methods are well-suited for high-cardinality categorical agricultural data.

14.1 Project Deliverables & Submission Links

• GitHub Repository: https://github.com/ShravaniGhadge18/crop-yield-prediction
• Live Deployed Web Application: https://crop-yield-prediction.streamlit.app
• Machine Learning Pipeline Script: crop_yield_model.py
• Interactive Web Application Script: app.py
• Jupyter Analysis & Modeling Notebook: crop.ipynb
• Serialized Production Models: models/best_model.pkl, random_forest_pipeline.pkl, decision_tree_pipeline.pkl, linear_regression_pipeline.pkl, linearsvr_pipeline.pkl
• Model Evaluation Artifacts: results/model_results.csv, results/best_model_info.json, results/model_comparison.png
• Comprehensive Technical Report: 125M1H018_FA2.pdf

15. Future Scope

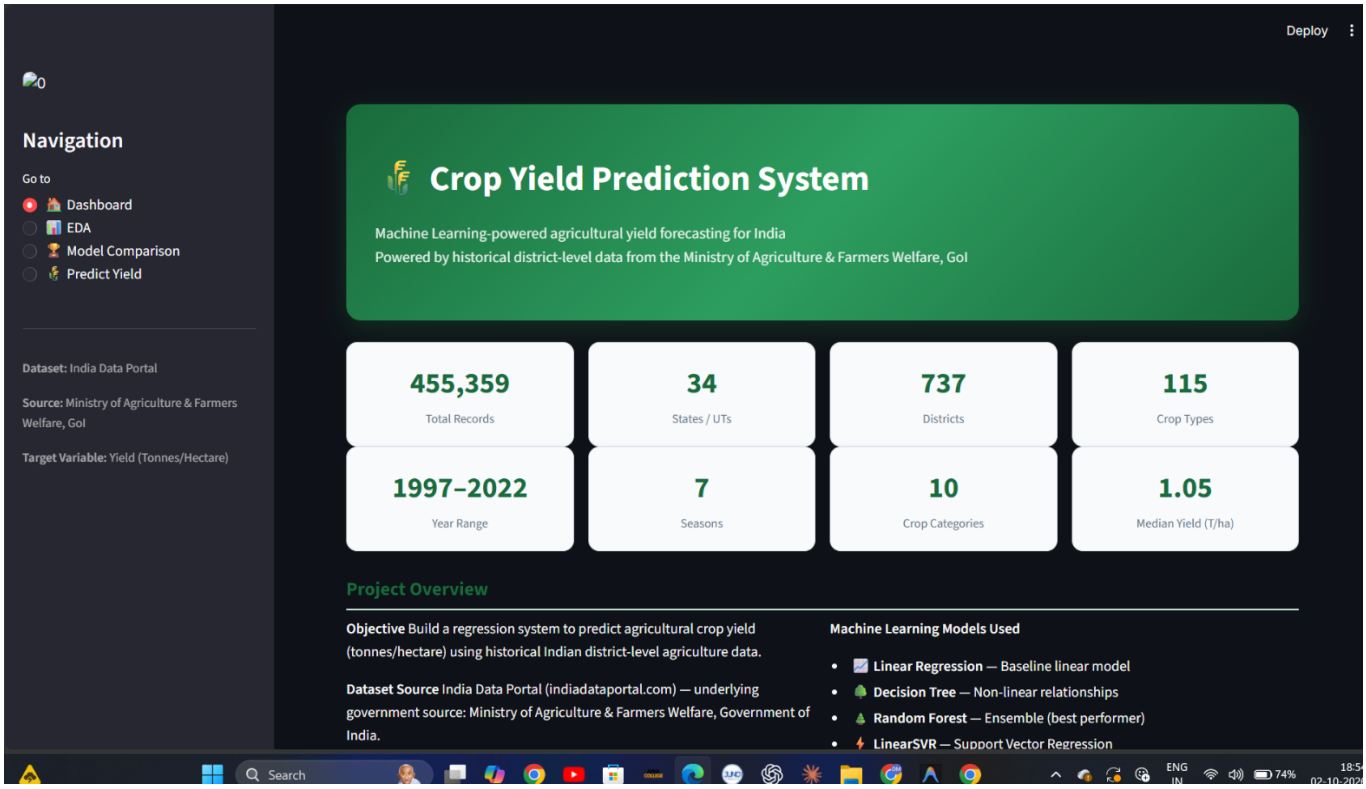
1. **Additional Features:** Incorporate rainfall, temperature, soil quality, and fertilizer data

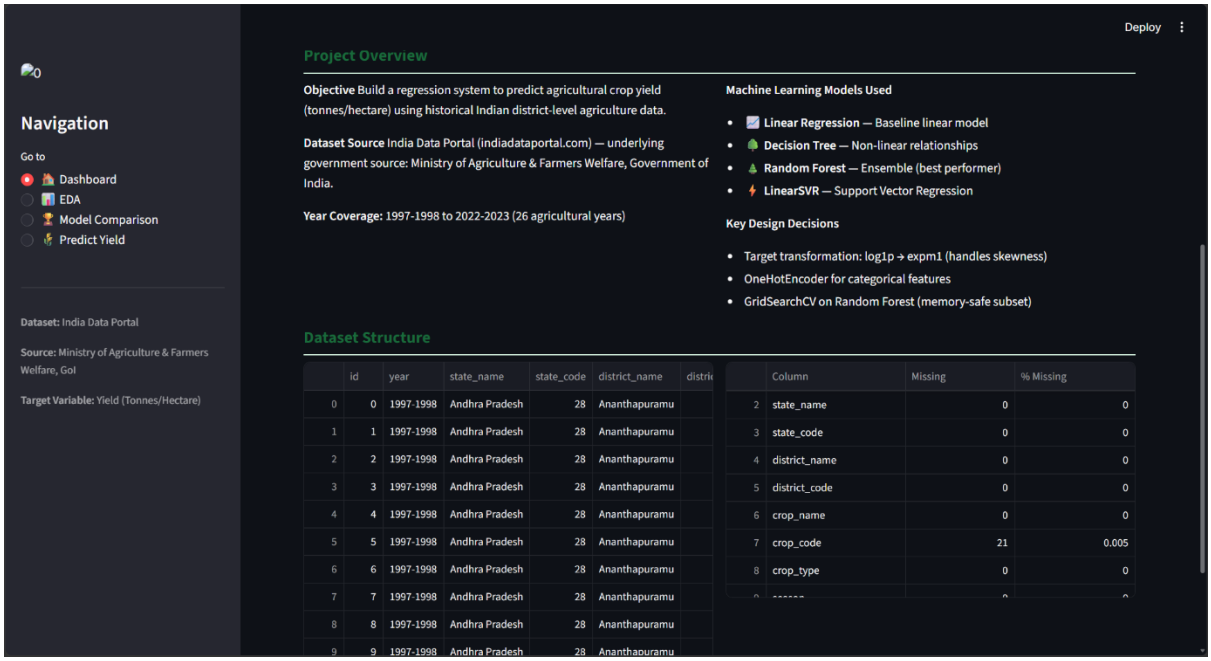
2. **Crop-Specific Models:** Train separate specialized models for high-yield crops like coconut
3. **Time Series Modeling:** Explore LSTM or Prophet for time-aware yield forecasting
4. **District-Level Models:** Hyperlocal models for each state/district
5. **Advanced Ensembles:** XGBoost, LightGBM, or stacking ensemble methods
6. **Better Deployment:** Docker containerization and cloud deployment (Heroku/Streamlit Cloud)

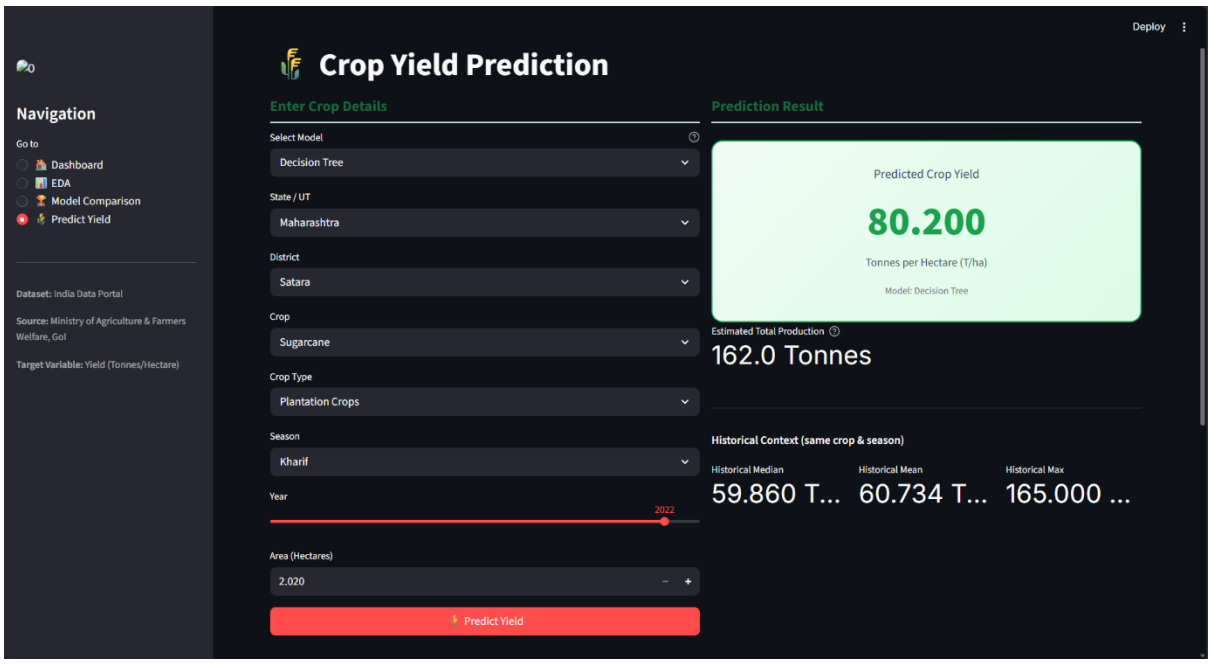
---

16. References

1. India Data Portal — Area, Production and Yield Data. https://indiadataportal.com/p/area-production-yield-apy
2. Ministry of Agriculture & Farmers Welfare, Government of India. *District Wise Crop Area and Production Data.*
3. Scikit-learn Documentation. *TransformedTargetRegressor.* https://scikit-learn.org/stable/modules/generated/sklearn.compose.TransformedTargetRegressor.html
4. Scikit-learn Documentation. *GridSearchCV.* https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
5. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR 12*, 2825–2830.
6. Breiman, L. (2001). Random Forests. *Machine Learning 45*, 5–32.
7. Vapnik, V. (1995). *The Nature of Statistical Learning Theory.* Springer.
8. Streamlit Documentation. https://docs.streamlit.io/


Deploy
Navigation
Go to
Dashboard
EDA
Model Comparison
Predict Yield
Dataset: India Data Portal
Source: Ministry of Agriculture & Farmers Welfare, GoI
Target Variable: Yield (Tonnes/Hectare)
Crop Yield Prediction System
Machine Learning-powered agricultural yield forecasting for India
Powered by historical district-level data from the Ministry of Agriculture & Farmers Welfare, GoI
455,359
Total Records
34
States / UTs
737
Districts
115
Crop Types
1997–2022
Year Range
7
Seasons
10
Crop Categories
1.05
Median Yield (T/ha)
Project Overview
Objective Build a regression system to predict agricultural crop yield (tonnes/hectare) using historical Indian district-level agriculture data.
Dataset Source India Data Portal (indiadataportal.com) — underlying government source: Ministry of Agriculture & Farmers Welfare, Government of India.
Machine Learning Models Used
Linear Regression — Baseline linear model
Decision Tree — Non-linear relationships
Random Forest — Ensemble (best performer)
LinearSVR — Support Vector Regression
Search
ENG
IN
74%


Deploy
Navigation
Go to
Dashboard
EDA
Model Comparison
Predict Yield
Dataset: India Data Portal
Source: Ministry of Agriculture & Farmers Welfare, GoI
Target Variable: Yield (Tonnes/Hectare)
Project Overview
Objective Build a regression system to predict agricultural crop yield (tonnes/hectare) using historical Indian district-level agriculture data.
Dataset Source India Data Portal (indiadataportal.com) — underlying government source: Ministry of Agriculture & Farmers Welfare, Government of India.
Year Coverage: 1997-1998 to 2022-2023 (26 agricultural years)
Machine Learning Models Used
Linear Regression — Baseline linear model
Decision Tree — Non-linear relationships
Random Forest — Ensemble (best performer)
LinearSVR — Support Vector Regression
Key Design Decisions
Target transformation: log1p → expm1 (handles skewness)
OneHotEncoder for categorical features
GridSearchCV on Random Forest (memory-safe subset)
Dataset Structure
id year state_name state_code district_name
0 0 1997-1998 Andhra Pradesh 28 Ananthapuramu
1 1 1997-1998 Andhra Pradesh 28 Ananthapuramu
2 2 1997-1998 Andhra Pradesh 28 Ananthapuramu
3 3 1997-1998 Andhra Pradesh 28 Ananthapuramu
4 4 1997-1998 Andhra Pradesh 28 Ananthapuramu
5 5 1997-1998 Andhra Pradesh 28 Ananthapuramu
6 6 1997-1998 Andhra Pradesh 28 Ananthapuramu
7 7 1997-1998 Andhra Pradesh 28 Ananthapuramu
8 8 1997-1998 Andhra Pradesh 28 Ananthapuramu
9 9 1997-1998 Andhra Pradesh 28 Ananthapuramu
Column Missing % Missing
2 state_name 0 0
3 state_code 0 0
4 district_name 0 0
5 district_code 0 0
6 crop_name 0 0
7 crop_code 21 0.005
8 crop_type 0 0


Deploy
Navigation
Go to
Dashboard
EDA
Model Comparison
Predict Yield
Dataset: India Data Portal
Source: Ministry of Agriculture & Farmers Welfare, GoI
Target Variable: Yield (Tonnes/Hectare)
Crop Yield Prediction
Enter Crop Details
Select Model
Decision Tree
State / UT
Maharashtra
District
Satara
Crop
Sugarcane
Crop Type
Plantation Crops
Season
Kharif
Year
2022
Area (Hectares)
2.020
Predict Yield
Prediction Result
Predicted Crop Yield
80.200
Tonnes per Hectare (T/ha)
Model: Decision Tree
Estimated Total Production
162.0 Tonnes
Historical Context (same crop & season)
Historical Median
59.860 T...
Historical Mean
60.734 T...
Historical Max
165.000 ...

Deploy

Navigation

Go to

- Dashboard
- EDA
- Model Comparison
- Predict Yield

Dataset: India Data Portal

Source: Ministry of Agriculture & Farmers Welfare, GoI

Target Variable: Yield (Tonnes/Hectare)

Season: Kharif

Year: 2022

Area (Hectares): 2.020

Predict Yield

Historical Context (same crop & season)

Historical Median	Historical Mean	Historical Max
59.860 T...	60.734 T...	165.000 ...

Input Summary

Parameter	Value
State	Maharashtra
District	Satara
Crop	Sugarcane
Crop Type	Plantation Crops
Season	Kharif
Year	2022
Area (ha)	2.02

Deploy

Navigation

Go to

- Dashboard
- EDA
- Model Comparison
- Predict Yield

Dataset: India Data Portal

Source: Ministry of Agriculture & Farmers Welfare, GoI

Target Variable: Yield (Tonnes/Hectare)

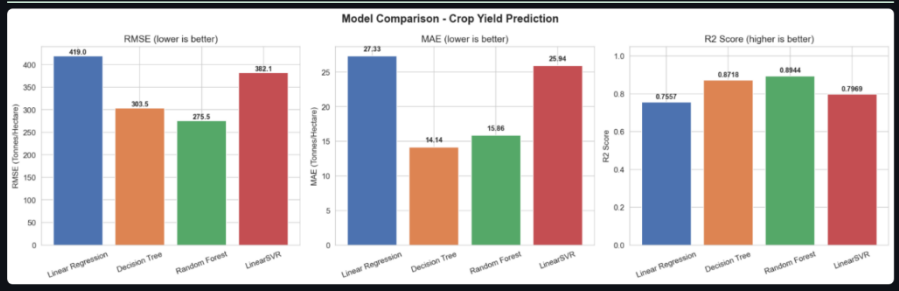
Model Comparison

Best Performing Model: **Random Forest**

Performance Metrics

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	27.328500	175586.800000	419.030800	0.755717
1	Decision Tree	14.140900	92118.430000	303.510200	0.871841
2	Random Forest	15.860400	75877.410000	275.458500	0.894437
3	LinearSVR	25.936000	145981.450000	382.075200	0.796905

Visual Comparison



GridSearchCV Results

Deploy

Navigation

Go to
- Dashboard
- EDA
- Model Comparison
- Predict Yield

Dataset: India Data Portal

Source: Ministry of Agriculture & Farmers Welfare, GoI

Target Variable: Yield (Tonnes/Hectare)



GridSearchCV Results

Best CV RMSE
552.4221

```
{
  "regressor__model__max_depth" : 15
  "regressor__model__n_estimators" : 50
}
```

Final MAE
15.8604

Final RMSE
275.4585

Final R2
0.894437

GridSearchCV Methodology Note: Due to the large dataset size (455,359 rows) and computational memory constraints, GridSearchCV was performed using a representative 10,000-row subset of the training data with 3-fold cross-validation (4 parameter combinations × 3 folds = 12 model trainings). The selected best hyperparameters were then used to retrain the Random Forest on the complete 80% training dataset (364,287 rows). The test set (91,072 rows) was never used during hyperparameter tuning.

Metric Explanations

MAE (Mean Absolute Error) Average absolute difference between predicted and actual yield. Measured in Tonnes/Hectare. Lower is better.

RMSE (Root Mean Squared Error) Square root of MSE — same unit as yield (T/ha). Lower is better.

MSE (Mean Squared Error) Average squared difference. Penalizes large errors more heavily. Lower is better.

R^2 Score Proportion of variance explained by the model. Range: 0 to 1. Higher is better (1 = perfect fit).

Deploy

Navigation

Go to
- Dashboard
- EDA
- Model Comparison
- Predict Yield

Dataset: India Data Portal

Source: Ministry of Agriculture & Farmers Welfare, GoI

Target Variable: Yield (Tonnes/Hectare)

Exploratory Data Analysis

Yield Statistics

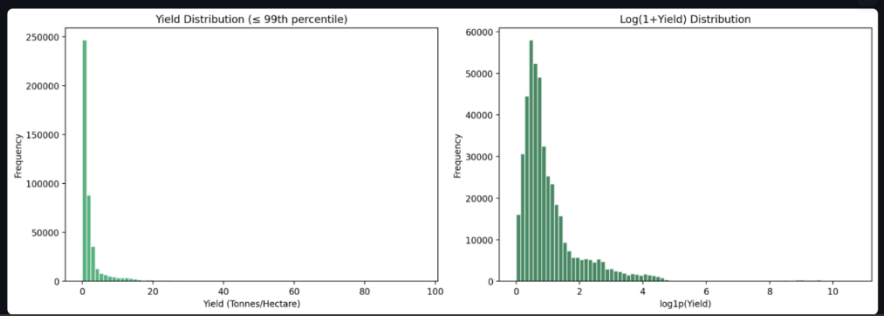
Min Yield	Median Yield	Mean Yield	Max Yield	Skewness
0.000	1.051	68.85	43958.3	16.33

log1p

Yield Distribution

Filter: Show yield below percentile

99



Deploy

Navigation

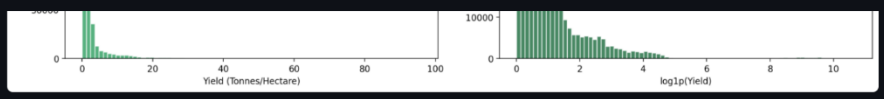
Go to

- Dashboard
- EDA
- Model Comparison
- Predict Yield

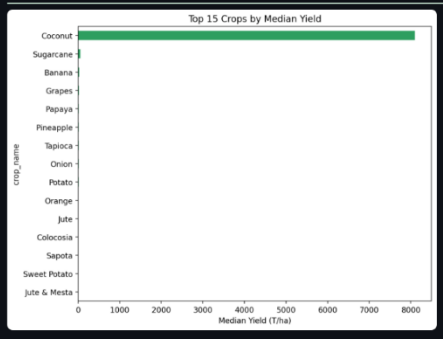
Dataset: India Data Portal

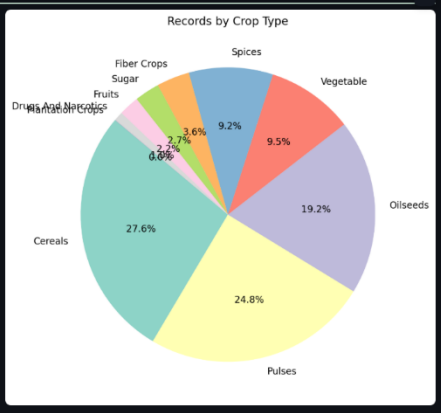
Source: Ministry of Agriculture & Farmers Welfare, GoI

Target Variable: Yield (Tonnes/Hectare)



Crop & Category Analysis





Temporal & Seasonal Trends





Deploy

Navigation

Go to

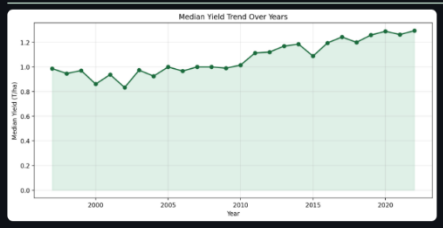
- Dashboard
- EDA
- Model Comparison
- Predict Yield

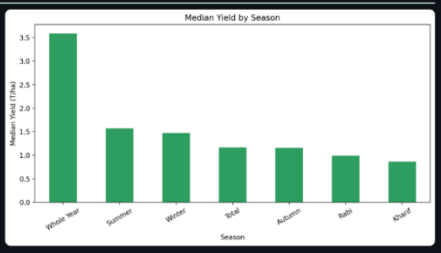
Dataset: India Data Portal

Source: Ministry of Agriculture & Farmers Welfare, GoI

Target Variable: Yield (Tonnes/Hectare)

Temporal & Seasonal Trends





State-wise Analysis

